

ARTIFICIAL INTELLIGENCE IN FINTECH / WEEK 04

From Signals to a Backtest

Week 3 asked what features and signals mean. Now trace a fixed signal through an assumed trade, the following holding interval, and the account value.

Issue	Question	Page
01	When can a signal start earning a return?	3
02	When does the first usable signal appear?	4
03	Why does this backtest shift the signal by two rows?	5
04	Does a signal of 1 mean another purchase?	6
05	Which stock returns belong to the strategy?	7
06	How do returns change the dollar balance?	8
07	Where do the shares and cash come from?	9
08	Why can the selling date still have a loss?	10
09	What goes wrong when a signal earns an earlier return?	11
10	Does next-open execution answer the same question?	12
11	How should buy and hold be aligned?	13
12	Are invested intervals the same as trades?	14
13	Do later observations change earlier results?	15
14	Can the last signal produce an observed return?	16
15	Does selling at the final mark change gross wealth?	17
16	Can daily OHLC tell which exit happened first?	18
17	Can a second calculation reproduce the backtest?	19

The main example uses AAPL, January 2-June 28, 2024. It is long or cash, with five-interval momentum, after-close signals and next-close execution. A signal does not earn returns that occurred before entry.

All account results are gross and price-only: no dividends, transaction costs, taxes or cash interest. Fractional shares are allowed. Prices are observations; the account and execution rules are assumptions.

Each issue answers a different question using the same data. Discussion is part of the lesson, not a new set of submissions. Week 5 adds prediction models, Week 6 costs, Week 7 performance evaluation and Week 8 weighting.

DATA & ACCOUNT ASSUMPTIONS

One dataset, one main trading rule

Begin with USD 1,000 in cash at January 2 Close. A positive five-interval Close return gives signal 1; otherwise 0. Signals require all six prices to be present. Remain in cash until a usable signal can be executed.

Assume each completed signal arrives in time for the next stored Close. Trade there to become fully invested or fully in cash. Do not add a trade at the final sample Close. Row offsets count stored observations, not calendar days; actual receipt and fills are not verified.

Before running: Which three Close values will locate the first entry and its following return?

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf

aapl = yf.Ticker("AAPL").history(
    start="2024-01-02", end="2024-07-01",
    interval="1d", auto_adjust=False, actions=True)
aapl.index = aapl.index.strftime("%Y-%m-%d")
assert aapl.index.is_unique and aapl.index.is_monotonic_increasing
prices = aapl[["Open", "High", "Low", "Close"]]
assert np.isfinite(prices).all().all() and (prices > 0).all().all()
close = aapl["Close"]
dates = ["2024-01-10", "2024-01-11", "2024-01-12"]
result = pd.DataFrame({"Date": dates,
    "Close (USD/share)": [f"{close[d]:.2f}" for d in dates]})
result
```

Date	Close (USD/share)
2024-01-10	186.19
2024-01-11	185.59
2024-01-12	185.92

The closes are USD 186.19, 185.59 and 185.92/share. The request returns 124 rows through June 28. Close is the provider price field, not a trade receipt; auto_adjust=False does not restore pre-split nominal prices.

Missing prices or unordered dates stop this example. The first row has no preceding requested Close; initial strategy return is zero because the account is cash, not because the stock return was observed to be zero.

Reuse these observations. Formulas use rounded inputs and approximate results; code retains full precision. Sources: [1], [2], [7].

ISSUE 01

When can a signal start earning a return?

A signal tells us what to do next. It cannot earn a price change that has already happened.

The first positive five-interval signal occurs after January 10 Close. Under the fixed rule, entry is assumed at January 11 Close, USD 185.59/share. The first holding interval then ends January 12 at USD 185.92/share.

Date	Event	Account consequence
Jan 10, after Close	Signal becomes 1	No completed purchase yet
Jan 11, at Close	Assumed purchase	Holding starts at USD 185.59
Jan 12, at Close	Next valuation	Holding marked at USD 185.92

The USD 186.19 January 10 Close helped form the signal. It is not the entry price. Using it as entry would give the account an earlier position than the written rule permits.

The January 11 close-to-close return began at January 10 Close, before the after-close signal existed. The new holding therefore does not earn that interval, even though the purchase date is January 11.

This lesson assumes execution at a published Close; it does not show an actual order filled there. A last-traded price is not a guaranteed execution price. The order type and delivery time would matter in a real implementation.

Discuss

1. For this January 10 signal, name the entry date and the first return-ending date. Explain why they differ.
2. If execution is delayed beyond January 11 Close, may this account still claim the entire January 11-12 return? Identify which boundary must be established again.

Sources: [2], [3]. Full titles and links are on the references page.

ISSUE 02

When does the first usable signal appear?

A missing signal is not the same as signal 0. The first means there is not enough history; the second is a completed calculation that does not meet the entry condition.

C is Close in USD/share. m is the decimal five-interval price return, so six dated closes are needed. Define signal s as 1 when m is positive, 0 when m is zero or negative, and missing before m can be calculated.

$$m_t = \frac{C_t}{C_{t-5}} - 1$$

$$m_{\text{Jan10}} = \frac{186.19}{184.25} - 1 \approx 0.010529 = 1.0529\%$$

January 10 uses January 3 Close, USD 184.25/share. January 9 is the first calculable date in this sample; its momentum is negative. The first five rows remain missing, not measured zeros.

Before running: Which date first has a calculated signal, and is that signal already 1?

```
momentum = close / close.shift(5) - 1
signal = pd.Series(np.nan, index=close.index)
valid = momentum.notna()
signal.loc[valid] = (momentum.loc[valid] > 0).astype(int)
dates = ["2024-01-08", "2024-01-09", "2024-01-10"]
result = pd.DataFrame({"Date": dates,
    "Momentum": ["Missing" if pd.isna(momentum[d])
        else f"{momentum[d]:.4%}" for d in dates],
    "Signal": ["Missing" if pd.isna(signal[d])
        else str(int(signal[d])) for d in dates]})
result
```

Date	Momentum	Signal
2024-01-08	Missing	Missing
2024-01-09	-0.2693%	0
2024-01-10	1.0529%	1

January 8 is missing. January 9 has -0.2693% and signal 0; January 10 has +1.0529% and signal 1. A rule condition, not a forecast fitted to future returns, produces these signals.

Discuss

1. Identify the two endpoint dates for the January 10 momentum. Why would only five prices be insufficient for this five-interval calculation?
2. Both January 8 and January 9 leave the account in cash for now. Why should the stored signal still distinguish missing from 0?

Sources: [3], [4]. Full titles and links are on the references page.

ISSUE 03

Why does this backtest shift the signal by two rows?

A position belongs to a holding interval. Here it records the fraction of the account invested from the previous Close to the current Close.

Let p_t be that fraction for the interval ending on observation t , and s_d the signal calculated after observation d . Execution is at $d+1$ Close; the first return ends at $d+2$. Setting $t=d+2$ gives $d=t-2$.

$$p_t = s_{t-2} \quad \text{when the controlling signal exists}$$

$$p_{\text{Jan12}} = s_{\text{Jan10}} = 1, \quad p_{\text{Jan11}} = s_{\text{Jan9}} = 0$$

p is either 0 for cash or 1 for full stock exposure, not a number of shares. The initial seven position rows are cash: five unavailable signal rows plus the two-row delay. This rule does not turn those missing signals into observed zeros.

Before running: Will the first invested return end January 11 or January 12?

```
position = signal.shift(2)
position.iloc[:7] = 0
assert position.notna().all() and position.isin([0, 1]).all()
labels = pd.Series(close.index, index=close.index)
signal_date = labels.shift(2)
interval_start = labels.shift(1)
dates = ["2024-01-11", "2024-01-12", "2024-01-16"]
result = pd.DataFrame({"Return end": dates,
    "Signal date": signal_date.loc[dates].to_list(),
    "Holding start": interval_start.loc[dates].to_list(),
    "Position": position.loc[dates].astype(int).to_list()})
result
```

Return end	Signal date	Holding start	Position
2024-01-11	2024-01-09	2024-01-10	0
2024-01-12	2024-01-10	2024-01-11	1
2024-01-16	2024-01-11	2024-01-12	1

January 12 is the first invested return. January 16 uses the January 11 signal, with exposure from January 12 Close. Stored observations jump across the weekend and holiday; `shift(2)` does not mean two calendar days.

The two-row lag follows from this execution rule. It is not a universal backtesting convention.

Discuss

1. Trace the January 16 row through all three dates in the table. Which Close begins its return?
2. If a dataset accidentally omitted a trading session, would `shift(2)` still mean two observed rows? Why would that not establish two consecutive exchange sessions?

Sources: [3]. Full titles and links are on the references page.

ISSUE 04

Does a signal of 1 mean another purchase?

Signal 1 requests a fully invested account. If the account is already fully invested, another signal 1 does not require another purchase.

p_t is the stock fraction held through the interval ending today. Let a_t be the fraction immediately after today's assumed closing trade. Normally a_t uses the previous stored observation's signal; at the final sample Close no new trade is added.

The change a minus p is +1 for cash-to-stock, -1 for stock-to-cash and 0 for no trade. These are changes in exposure, not share quantities or dollar turnover.

$$\text{Closing change} = a_t - p_t$$

$$\text{Jan11: } 1 - 0 = 1, \quad \text{Jan12: } 1 - 1 = 0, \quad \text{Jan17: } 0 - 1 = -1$$

Before running: Which of these three closing dates has no trade despite an invested account?

```
after_close = signal.shift(1)
after_close.iloc[:6] = 0
assert after_close.notna().all()
after_close.iloc[-1] = position.iloc[-1] # No final new trade.
exposure_change = after_close - position
dates = ["2024-01-11", "2024-01-12", "2024-01-17"]
result = pd.DataFrame({"Trade date": dates,
    "Before": position.loc[dates].astype(int).to_list(),
    "After": after_close.loc[dates].astype(int).to_list(),
    "Change": exposure_change.loc[dates].astype(int).to_list()})
result
```

Trade date	Before	After	Change
2024-01-11	0	1	1
2024-01-12	1	1	0
2024-01-17	1	0	-1

January 11 is an entry. January 12 stays fully invested with change 0. January 17 is an exit. Use the execution date for this trade table, not the following return-ending date.

Discuss

1. Why does the January 12 row have position 1 but no purchase? Identify the difference between a requested state and a change of state.
2. Does change -1 mean selling exactly one share? What account value and price information is needed to calculate the share quantity?

Sources: [3]. Full titles and links are on the references page.

ISSUE 05

Which stock returns belong to the strategy?

A stock return belongs to the strategy only if the account held that stock over the same interval.

r_t is the decimal AAPL Close-to-Close return ending on t . p_t is the stock fraction held during that interval. $r_{p,t}^{\text{gross}}$ is the account return before costs. Cash earns zero and dividends are excluded.

For each beginning dollar, p is invested and $1-p$ stays in cash. The ending value is $p(1+r)+(1-p)=1+pr$; subtract the beginning value of 1 to obtain pr .

$$r_t = \frac{C_t}{C_{t-1}} - 1, \quad r_{p,t}^{\text{gross}} = p_t r_t$$

$$r_{p,\text{Jan12}}^{\text{gross}} = 1 \left(\frac{185.92}{185.59} - 1 \right) \approx 0.1778\%$$

Before running: Will the cash account earn AAPL's positive January 18 return?

```
asset_return = close.pct_change(fill_method=None)
exposed = position.eq(1)
assert np.isfinite(asset_return.loc[exposed]).all()
gross_return = pd.Series(0.0, index=close.index)
gross_return.loc[exposed] = asset_return.loc[exposed]
dates = ["2024-01-12", "2024-01-17", "2024-01-18"]
result = pd.DataFrame({"Return end": dates,
    "Position": position.loc[dates].astype(int).to_list(),
    "AAPL": [f"{asset_return[d]:.4%}" for d in dates],
    "Strategy": [f"{gross_return[d]:.4%}" for d in dates]})
result
```

Return end	Position	AAPL	Strategy
2024-01-12	1	0.1778%	0.1778%
2024-01-17	1	-0.5174%	-0.5174%
2024-01-18	0	3.2571%	0.0000%

January 12 gives +0.1778% to both stock and strategy. January 17 gives -0.5174% to both. On January 18 AAPL gains 3.2571%, but position 0 gives the strategy 0%. Gross can be negative; it means before costs, not profit.

The first asset return stays missing. Strategy return there is zero only because initial cash earns zero. A missing exposed return must stop calculation, not be replaced with zero.

Discuss

1. Why is the January 18 strategy return zero despite the stock gain? Name the position and the exact holding interval.
2. Why is initial cash compatible with a missing first asset return, while a later invested interval with an unknown endpoint cannot use the same zero treatment?

Sources: [4]. Full titles and links are on the references page.

ISSUE 06

How do returns change the dollar balance?

Returns describe relative changes. Wealth shows what those changes do to the same account balance over time.

W_t is account value in USD at Close t . Start at $W_0 = \text{USD } 1,000$ on January 2. With no deposits, withdrawals, costs or distributions, multiply the previous balance by one plus the current strategy return.

$$W_t = W_{t-1}(1 + r_{p,t}^{\text{gross}})$$

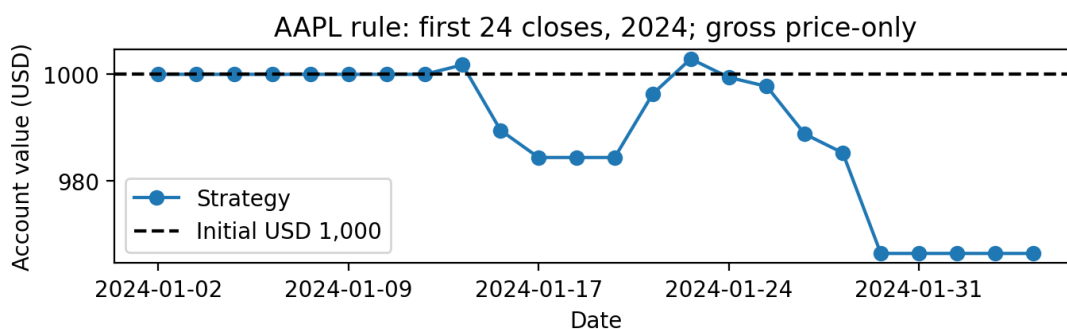
$$W_{\text{Jan12}} \approx 1000(1 + 0.001778) \approx 1001.78 \text{ USD}$$

$$W_{\text{Jan16}} \approx 1001.78(1 - 0.012317) \approx 989.44 \text{ USD}$$

The January 16 loss applies to USD 1,001.78, not to a new USD 1,000 account. Flat sections mean the modeled balance did not change; they do not identify whether cash or an unchanged stock price caused it.

Before running: After the small January 12 gain, will the January 16 loss leave the account above or below USD 1,000?

```
wealth = 1000 * (1 + gross_return).cumprod(skipna=False)
assert wealth.notna().all() and (wealth > 0).all()
fig, ax = plt.subplots(figsize=(7, 2.3))
wealth.iloc[:24].plot(ax=ax, marker="o", label="Strategy")
ax.axhline(1000, color="black", linestyle="--", label="Initial USD 1,000")
ax.set(title="AAPL rule: first 24 closes, 2024; gross price-only",
       xlabel="Date", ylabel="Account value (USD)")
ax.legend()
plt.tight_layout()
```



The balance rises to USD 1001.78 on January 12, then falls to USD 989.44 on January 16. It is USD 984.32 on January 17. This is a modeled gross price-only path, not a cash statement from a broker.

Discuss

1. Use the January 12 balance and January 16 return to explain the January 16 dollar loss. Why is the percentage not applied to a fresh initial balance?
2. With fractional shares and all other rules fixed, would doubling initial capital change the percentage return path or only dollar values? Explain using the recurrence.

Sources: [5]. Full titles and links are on the references page.

ISSUE 07

Where do the shares and cash come from?

A dollar return calculation should agree with an account containing shares and cash. Buying exchanges cash for shares; it does not create wealth.

q_t is shares held after the closing trade, B_t is cash in USD, and a_t is the after-trade stock fraction. C_t is USD/share and W_t is USD. Fractional shares are allowed; no costs or outside cash flows occur.

$$q_t = \frac{a_t W_t}{C_t}, \quad B_t = (1 - a_t) W_t, \quad W_t = q_t C_t + B_t$$

$$q_{\text{Jan11}} = \frac{1(1000)}{185.59} \approx 5.38822 \text{ shares}, \quad B_{\text{Jan11}} = 0$$

The January 11 purchase converts the USD 1,000 cash into stock. During continued full investment, the share quantity stays the same even though its dollar value changes. On exit, the shares are converted back to cash.

Before running: Will shares increase on January 12 simply because the stock price and wealth increased?

```
shares_after = after_close * wealth / close
cash_after = (1 - after_close) * wealth
share_trade = shares_after - shares_after.shift(1).fillna(0)
assert np.allclose(shares_after * close + cash_after, wealth)
dates = ["2024-01-11", "2024-01-12", "2024-01-17"]
result = pd.DataFrame({"After Close": dates,
    "Shares": [f"{shares_after[d]:.5f}" for d in dates],
    "Cash (USD)": [f"{cash_after[d]:.2f}" for d in dates],
    "Wealth (USD)": [f"{wealth[d]:.2f}" for d in dates]})
result
```

After Close	Shares	Cash (USD)	Wealth (USD)
2024-01-11	5.38822	0.00	1000.00
2024-01-12	5.38822	0.00	1001.78
2024-01-17	0.00000	984.32	984.32

Shares remain 5.38822 on January 11 and 12. January 17 has zero shares and USD 984.32 cash after exit. The loss was not caused by converting stock to cash at the same valuation price; it arose while the shares were held.

Discuss

1. Reconstruct January 12 wealth from the share quantity and Close. Why is buying more shares unnecessary to keep this account fully invested?
2. If only whole shares were allowed, could the January 11 account buy 5.38822 shares? Explain which remaining-cash assumption and return calculation would need to change.

Sources: [8]. Full titles and links are on the references page.

ISSUE 08

Why can the selling date still have a loss?

Selling at the end of an interval does not remove the gain or loss already earned while holding the shares during that interval.

AAPL falls from January 16 Close, USD 183.63/share, to January 17 Close, USD 182.68/share. The account still holds 5.38822 shares through that interval; its exit is assumed at January 17 Close.

q_{t-1} is the shares held after the previous Close. With no intermediate trade or cash interest, the current dollar gain or loss is that share quantity times the Close change.

$$W_t - W_{t-1} = q_{t-1}(C_t - C_{t-1})$$

$$W_{\text{Jan17}} - W_{\text{Jan16}} \approx 5.38822(182.68 - 183.63) \approx -5.12 \text{ USD}$$

Before running: Will the January 17 return become zero because its after-close holding is zero?

```
dates = ["2024-01-16", "2024-01-17", "2024-01-18"]
result = pd.DataFrame({"Date": dates,
    "Held in interval": position.loc[dates].astype(int).to_list(),
    "Held after Close": after_close.loc[dates].astype(int).to_list(),
    "Strategy return": [f"{gross_return[d]:.4%}" for d in dates]})
result
```

Date	Held in interval	Held after Close	Strategy return
2024-01-16	1	1	-1.2317%
2024-01-17	1	0	-0.5174%
2024-01-18	0	0	0.0000%

January 17 has interval position 1, after-close position 0 and return -0.5174%. Wealth falls from USD 989.44 to USD 984.32 before the stock is exchanged for cash. January 18 then has zero exposure and zero strategy return.

Using the after-close fraction to explain the interval just finished confuses the ending account with the account that earned the return.

Discuss

1. Calculate the result of multiplying January 17 AAPL return by its after-close fraction. Which actual modeled loss would that wrong calculation erase?
2. If the sale were postponed until January 18 Close, would the January 17 holding loss disappear? Explain which interval would gain additional exposure instead.

Sources: [4]. Full titles and links are on the references page.

ISSUE 09

What goes wrong when a signal earns an earlier return?

An early signal alignment is wrong because it violates the execution rule, not because it necessarily produces a larger return.

Keep the same prices, five-interval signal, cash rule and assumed next-close execution. Compare three calculations: signal from the current row, one row earlier, and the required two rows earlier. Only the last obeys the written rule.

These are timing tests on unchanged market data. The first two columns deliberately implement the rule incorrectly; they are not alternative strategies selected for their results.

Before running: Will the incorrect one-row version gain or lose on January 11? Does its sign determine whether the timing is allowed?

```
same_row_return = signal.fillna(0) * asset_return
one_row_return = signal.shift(1).fillna(0) * asset_return
dates = ["2024-01-10", "2024-01-11", "2024-01-12"]
result = pd.DataFrame({"Return end": dates,
    "No shift": [f"{same_row_return[d]:.4%}" for d in dates],
    "One row": [f"{one_row_return[d]:.4%}" for d in dates],
    "Rule: two rows": [f"{gross_return[d]:.4%}" for d in dates]})
result
```

Return end	No shift	One row	Rule: two rows
2024-01-10	0.5671%	0.0000%	0.0000%
2024-01-11	-0.3223%	-0.3223%	0.0000%
2024-01-12	0.1778%	0.1778%	0.1778%

On January 10 the no-shift version claims +0.5671% before a purchase could occur. On January 11 the one-row version claims -0.3223% for the pre-entry interval. Both violate timing. The correctly timed strategy has 0% on both dates.

All three produce +0.1778% on January 12 because the relevant signals happen to agree. A matching row therefore cannot validate the entire alignment.

Discuss

1. For the January 11 one-row result, name the signal date and the incorrectly assigned interval. Explain why even a negative result is inadmissible.
2. Why does agreement on January 12 not validate the no-shift code? Identify an earlier date that distinguishes the calculations.

Sources: [3]. Full titles and links are on the references page.

ISSUE 10

Does next-open execution answer the same question?

Changing the assumed execution price also changes the holding interval. The same signal can therefore lead to returns with opposite signs.

Keep the January 10 after-close signal. Compare a next-open entry with exit at that day's Close, against the main next-close entry with valuation one Close later. The first is an explicitly different execution and holding rule, not a correction to the main backtest.

O_t and C_t are that date's Open and Close, in USD/share. The superscript O-to-C identifies an intraday price return; the main r remains Close-to-Close.

$$r_t^{O \rightarrow C} = \frac{C_t}{O_t} - 1$$

$$r_{\text{Jan11}}^{O \rightarrow C} = \frac{185.59}{186.54} - 1 \approx -0.5093\%$$

$$r_{\text{Jan12}} = \frac{185.92}{185.59} - 1 \approx +0.1778\%$$

Before running: Will these two returns have the same sign? Which one belongs to the main backtest?

```
open_jan11 = aapl.loc["2024-01-11", "Open"]
intraday_return = close["2024-01-11"] / open_jan11 - 1
following_return = close["2024-01-12"] / close["2024-01-11"] - 1
result = pd.DataFrame({
    "Interval": ["Jan 11 Open-Close", "Jan 11 Close-Jan 12 Close"],
    "Entry (USD)": [f"{open_jan11:.2f}", f"{close['2024-01-11']:.2f}"],
    "Return": [f"{intraday_return:.4%}", f"{following_return:.4%}"]}
result
```

Interval	Entry (USD)	Return
Jan 11 Open-Close	186.54	-0.5093%
Jan 11 Close-Jan 12 Close	185.59	0.1778%

The open-to-close return is -0.5093%; the following close-to-close return is +0.1778%. The opposite signs come from different price endpoints and exposure periods. The main backtest keeps the latter rule throughout.

Discuss

1. Identify each entry and exit or valuation price. Why would inserting the intraday return into the main ledger change the strategy?
2. Could a method comparison attribute these opposite signs to a better signal model? State which execution and holding conditions would first need to be matched.

Sources: [2], [4]. Full titles and links are on the references page.

ISSUE 11

How should buy and hold be aligned?

A benchmark needs the same valuation period and price convention. It answers what happened under a simple alternative, not whether the tested rule will work in the future.

Buy and hold is assumed to invest the same USD 1,000 at January 2 Close and keep its shares through June 28. Unlike the signal rule, this prespecified benchmark does not wait for momentum. Both include the initial date and use price-only returns, zero costs and no final sale.

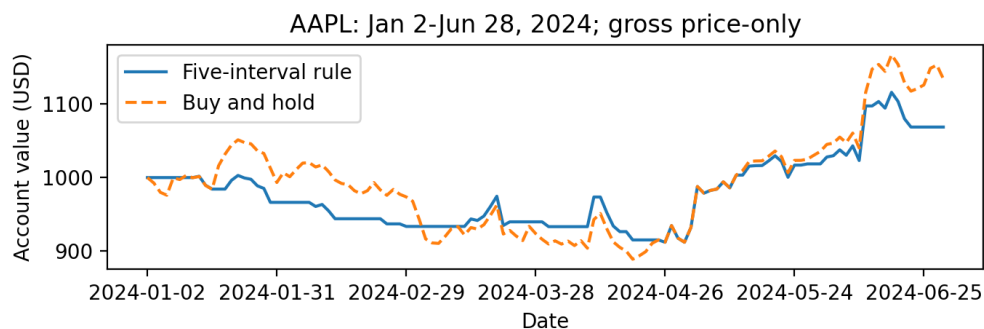
W_t^{BH} is benchmark wealth in USD. Fixed shares equal initial capital divided by January 2 Close, C_0 ; valuing them at C_t gives the ratio below.

$$W_t^{BH} = 1000 \frac{C_t}{C_0}$$

$$W_{\text{Jan3}}^{BH} = 1000 \frac{184.25}{185.64} \approx 992.51 \text{ USD}$$

Before running: Does buy and hold remain flat while the signal rule waits in initial cash?

```
buy_hold = 1000 * close / close.iloc[0]
assert buy_hold.index.equals(wealth.index)
fig, ax = plt.subplots(figsize=(7, 2.5))
wealth.plot(ax=ax, label="Five-interval rule")
buy_hold.plot(ax=ax, label="Buy and hold", linestyle="--")
ax.set(title="AAPL: Jan 2-Jun 28, 2024; gross price-only",
        xlabel="Date", ylabel="Account value (USD)")
ax.legend()
plt.tight_layout()
```



Buy and hold falls to USD 992.51 on January 3 while the rule remains at USD 1,000. By June 28 their balances are USD 1134.56 and USD 1068.80, respectively. The comparison includes all initial cash intervals.

These endpoints describe this sample under these assumptions. They do not include dividends, execution costs, risk or uncertainty; those omissions prevent a general investment ranking.

Discuss

1. If someone starts the benchmark on January 12 but leaves the rule starting January 2, which comparison condition has changed? Explain why both dates must be explicit.
2. Does a higher June 28 balance establish that the benchmark has less risk? Identify what this endpoint alone cannot describe about the intervening path.

Sources: [4], [5]. Full titles and links are on the references page.

ISSUE 12

Are invested intervals the same as trades?

Holding stock for several intervals is different from buying it several times. Count exposure intervals and trading events separately.

The first entry is January 11 and the first exit January 17. Stock is held over three intervals ending January 12, 16 and 17. There is one purchase and one sale, not three purchases.

Before running: Will the first holding period contain more invested intervals than purchases?

```

windows = [("2024-01-11", "2024-01-17"),
            ("2024-01-02", "2024-06-28")]
records = []
for first, last in windows:
    # Trades include the opening boundary; returns start after it.
    held = position.loc[first:last].iloc[1:]
    changes = exposure_change.loc[first:last]
    records.append([first, last, int(held.eq(1).sum()),
                    int(changes.eq(1).sum()), int(changes.eq(-1).sum())])
result = pd.DataFrame(records, columns=[
    "From Close", "To Close", "Held intervals", "Entries", "Exits"])
result

```

From Close	To Close	Held intervals	Entries	Exits
2024-01-11	2024-01-17	3	1	1
2024-01-02	2024-06-28	63	15	15

The first period has three invested intervals, one entry and one exit. Across the full sample there are 63 invested intervals, 15 entries and 15 exits. These counts have different meanings.

An entry count does not report shares or dollars traded. The change from cash to full investment may involve a different amount each time because account wealth changes.

Discuss

1. Name the three first-period return intervals and the two trade dates. Why is counting every positive position as a purchase wrong?
2. Two entries occur at different prices and account balances. Could they have equal event counts but different dollar amounts? Identify the inputs needed to calculate each amount.

Sources: [3]. Full titles and links are on the references page.

ISSUE 13

Do later observations change earlier results?

Adding later observations should not change earlier signals or earned returns for this fixed rule when the earlier input values are unchanged.

Recalculate using only the actual observations through January 17. Do not edit prices or fill gaps. Compare the earlier signals, interval positions and wealth with the full-sample calculation. A trade added or omitted at a newly chosen final boundary is a separate terminal rule, not part of this comparison.

Before running: Will adding January 18-June 28 prices change any of these three earlier balances?

```
short_close = close.loc["2024-01-17"]
short_m = short_close / short_close.shift(5) - 1
short_signal = (short_m > 0).astype(float).where(short_m.notna())
short_position = short_signal.shift(2)
short_position.iloc[:7] = 0
short_return = short_close.pct_change(fill_method=None)
assert np.isfinite(short_return.loc[short_position.eq(1)]).all()
short_gross = short_return.where(short_position.eq(1), 0)
short_wealth = 1000 * (1 + short_gross).cumprod(skipna=False)
assert np.allclose(short_signal, signal.loc[short_close.index], equal_nan=True)
assert np.allclose(short_position, position.loc[short_close.index])
assert np.allclose(short_wealth, wealth.loc[short_close.index])
dates = ["2024-01-12", "2024-01-16", "2024-01-17"]
result = pd.DataFrame({"Date": dates,
    "Short sample (USD)": [f"{short_wealth[d]:.2f}" for d in dates],
    "Full sample (USD)": [f"{wealth[d]:.2f}" for d in dates]})
result
```

Date	Short sample (USD)	Full sample (USD)
2024-01-12	1001.78	1001.78
2024-01-16	989.44	989.44
2024-01-17	984.32	984.32

Both calculations show USD 1,001.78, USD 989.44 and USD 984.32 on the three dates. The numerical checks cover all overlapping rows, not only the displayed sample.

This checks whether this implementation uses later observations for earlier results. It does not verify historical provider versions or prove that every possible information-timing error is absent.

Discuss

1. If January 12 position changed after later prices were added, which calculations would you inspect first? Relate your answer to trailing inputs, shifts and any full-sample fitting.
2. If the provider revised January 12 Close between downloads, would this still be a comparison with unchanged earlier inputs? Explain which difference must be separated from code using future observations.

Sources: [3], [8]. Full titles and links are on the references page.

ISSUE 14

Can the last signal produce an observed return?

A signal near the end of a dataset may not have enough later prices to produce an observed holding return.

The June 27 after-close signal is 1. Its usual next-close execution would be June 28, but the following Close needed to value a full holding interval is absent. The June 28 signal lacks even a next stored execution Close.

Our terminal rule adds no new trade at the final sample Close. It keeps the account in cash on June 28 despite the recent positive signals. Do not treat an unobserved future return as zero stock performance.

Before running: Will either final positive signal create an invested interval inside this sample?

```
dates = ["2024-06-26", "2024-06-27", "2024-06-28"]
result = pd.DataFrame({"Date": dates,
    "After-close signal": signal.loc[dates].astype(int).to_list(),
    "Interval position": position.loc[dates].astype(int).to_list(),
    "Closing change": exposure_change.loc[dates].astype(int).to_list(),
    "Wealth (USD)": [f"{wealth[d]:.2f}" for d in dates]})
result
```

Date	After-close signal	Interval position	Closing change	Wealth (USD)
2024-06-26	0	0	0	1068.80
2024-06-27	1	0	0	1068.80
2024-06-28	1	0	0	1068.80

June 27 and 28 have signal 1, but all three displayed interval positions are 0. The balance remains USD 1068.80. That flat balance follows from the cash and terminal rules, not evidence that later AAPL returns are zero.

Discuss

1. For the June 27 signal, which execution price exists and which later valuation price is missing? Why is a complete earned return unavailable?
2. If the backtest were extended with later observations, which pending signal could then be evaluated? Explain why this would extend the test period rather than fill a missing result with zero.

Sources: [1], [3]. Full titles and links are on the references page.

ISSUE 15

Does selling at the final mark change gross wealth?

An account can have the same gross value before and after a sale but a different mix of shares and cash.

For this comparison only, end valuation on January 12 while the first holding is still long. Compare retaining the shares with a prearranged sale at that same Close, USD 185.92/share. No additional return interval or cost is added. This does not change the main June 28 backtest.

q is the shares before sale, B is cash and W is total value. Primes indicate the quantities after the hypothetical sale. At the identical price C and zero costs, cash increases by exactly the stock value removed.

$$q' = 0, \quad B' = B + qC, \quad W' = B' = W$$

$$B' \approx 0 + 5.38822(185.92) \approx 1001.78 \text{ USD}$$

Before running: Will the sale change total gross value or only the form in which it is held?

```
end_date = "2024-01-12"
q = shares_after[end_date]
mark = close[end_date]
marked_value = q * mark
result = pd.DataFrame({
    "At Jan 12 Close": ["Retain shares", "Sell at same mark"],
    "Shares after": [f"{q:.5f}", "0.00000"],
    "Cash (USD)": [f"0.00", f"{marked_value:.2f}"],
    "Wealth (USD)": [f"{marked_value:.2f}", f"{marked_value:.2f}"]}
result
```

At Jan 12 Close	Shares after	Cash (USD)	Wealth (USD)
Retain shares	5.38822	0.00	1001.78
Sell at same mark	0.00000	1001.78	1001.78

Both rows have USD 1,001.78. The first retains 5.38822 shares; the second has zero shares and cash. The sale adds a trading event even though the gross endpoint is unchanged.

Equality depends on the same marking and execution price with zero costs. A sale later at an unknown price need not preserve value. An actual order cannot be assumed to fill at the mark merely because the accounting balances.

Discuss

1. Why can a wealth-only report fail to reveal whether this final sale occurred? Identify the additional holding or trade information required.
2. If the reported sale price differs from the marking price, which equality above must be recalculated? Explain why labeling the added row zero-return would be insufficient.

Sources: [2]. Full titles and links are on the references page.

ISSUE 16

Can daily OHLC tell which exit happened first?

A daily bar gives price extremes but not the order in which they occurred. A backtest must not silently turn that missing sequence into an observed trade result.

Use AAPL on January 3. For this illustration only, assume entry at Open, USD 184.22/share, followed by a stop-loss instruction at USD 184 and a take-profit limit at USD 185. The two levels are teaching settings, not actual orders or changes to the main strategy.

Before running: Will the daily Low and High show that both specified levels lie inside the range? Will they show which was reached first?

```
bar = aapl.loc[["2024-01-03"], ["Open", "High", "Low", "Close"]]
result = bar.rename(columns={
    "Open": "Open (USD)", "High": "High (USD)",
    "Low": "Low (USD)", "Close": "Close (USD)"})
    .map(lambda value: f"{value:.2f}")
    .reset_index(names="Date")
result
```

Date	Open (USD)	High (USD)	Low (USD)	Close (USD)
2024-01-03	184.22	185.88	183.43	184.25

Open is USD 184.22, High USD 185.88, Low USD 183.43 and Close USD 184.25/share. The USD 184 lower level and USD 185 upper level both lie inside the range. Neither the closing value nor the daily extremes reveal which level was reached first after entry.

A platform may choose an intrabar path assumption or use finer data. An assumed path is not evidence of the historical sequence. Even a recorded crossing does not by itself establish the execution price: a triggered stop becomes a market order.

Do not calculate a single realized profit from this daily row. Finer event ordering and an explicit order/fill model are needed; finer bars may still omit order-level detail.

Discuss

1. Using the displayed OHLC values, explain why both levels could matter while the daily row cannot select the first exit. Which additional observation would help distinguish them?
2. A backtest automatically chooses the profitable exit whenever both levels lie within the bar. What unsupported assumption did it insert, and how should the result be labeled or withheld?

Sources: [2], [6]. Full titles and links are on the references page.

ISSUE 17

Can a second calculation reproduce the backtest?

Check the account by valuing shares and cash directly, rather than multiplying the return series again.

At each Close, first value the existing shares and cash. Only then apply the previous stored signal to the assumed closing trade. The last Close is valuation only. This follows the same rules as the main backtest through a different arithmetic route.

Before running: Will direct shares-plus-cash valuation match the return-product balance on every date, within rounding tolerance?

```
cash, shares = 1000.0, 0.0
balances = []
for i, price in enumerate(close):
    value = cash + shares * price
    balances.append(value)
    if i == len(close) - 1:
        continue # No new trade at the final Close.
    previous_signal = signal.iloc[i-1] if i > 0 else np.nan
    target = 0 if pd.isna(previous_signal) else int(previous_signal)
    desired_shares = value / price if target == 1 else 0.0
    cash -= (desired_shares - shares) * price
    shares = desired_shares
loop_wealth = pd.Series(balances, index=close.index)
assert np.allclose(loop_wealth, wealth, rtol=1e-10, atol=1e-8)
dates = ["2024-01-12", "2024-01-17", "2024-06-28"]
result = pd.DataFrame({"Date": dates,
    "Return product (USD)": [f"{wealth[d]:.2f}" for d in dates],
    "Shares + cash (USD)": [f"{loop_wealth[d]:.2f}" for d in dates]})
result
```

Date	Return product (USD)	Shares + cash (USD)
2024-01-12	1001.78	1001.78
2024-01-17	984.32	984.32
2024-06-28	1068.80	1068.80

The two routes agree at USD 1,001.78, USD 984.32 and USD 1,068.80 on the displayed dates. The assertion checks all 124 balances. It would stop execution if they differed beyond the stated tolerance.

Agreement supports the accounting under these rules. Both calculations still share prices and execution assumptions; neither establishes real fills or profitable future performance. The January 10 signal, January 11 entry and January 12 return still need the separate timing check from Issue 1.

Discuss

1. If both calculations accidentally used the January 10 Close for entry, could matching balances detect that shared error? Use the three dated events above to specify a check that could.
2. When Week 5 replaces the rule with model predictions, which information time and entry/return dates must remain explicit? Explain why accurate predictions alone cannot verify the account balance.

Sources: [2], [8]. Full titles and links are on the references page.

REFERENCES & REPRODUCTION

Sources and calculation notes

- [1] [yfinance](#): historical prices
- [2] [Investor.gov](#): order types and execution prices
- [3] [pandas](#): shift observations
- [4] [pandas](#): fractional price changes
- [5] [pandas](#): cumulative product
- [6] [TradingView](#): broker-emulator intrabar assumptions
- [7] [Yahoo Finance](#): AAPL history and price-field notes
- [8] [NumPy](#): numerical comparisons

What these sources establish

Yahoo Finance provides the price observations accessed through [yfinance](#). Software documentation explains row shifts and arithmetic. [Investor.gov](#) distinguishes order instructions from guaranteed prices. [TradingView](#) documents assumptions used when intrabar information is absent; this lesson does not replicate its emulator.

The financial equations are derived from the stated holdings and cash account. The next-close rule, zero cash interest, fractional shares and omitted costs/dividends are teaching assumptions, not observed trades or recommended parameters.

Executed 2026-09-14 UTC: Python 3.12.9, numpy 2.2.6, pandas 2.2.3, matplotlib 3.10.8, yfinance 1.2.0. Run the displayed code in order in a fresh Python session with these packages installed. Internet and provider availability are required.

The full downloaded observations and verification records stay private. Historical responses can change. No historical receipt log, real fill or point-in-time provider version was verified. No risk-adjusted ranking or future-performance claim is made.